Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1

По дисциплине «Модели решения задач в интеллектуальных системах»

Тема: «Реализация многослойного персептрона для решения задачи «исключающее ИЛИ»

Выполнил:
Студент 4 курса
Группы ИИ-26
Турич Д.А.
Проверил:
Никитин С. К.

Брест 2026

Цель работы

Реализовать персептроны с сигмоидной активацией и обучением методом обратного распространения ошибки в режиме online, исследовать XOR для числовых кодов классов и сравнить однослойную и многослойную архитектуры.

1. Условие и индивидуальный вариант №6

Для варианта 6 заданы классы $c_0 = -9$ и $c_1 = 6$. Задача XOR определяет значение c_1 при разных классах двух входов и c_0 при одинаковых.

A	B	A ⊕B
-9	-9	-9
-9	6	6
6	-9	6
6	6	-9

2. Архитектура и метод

Многослойный персептрон MLP 2–2–1: два входа, два сигмоидных нейрона скрытого слоя, один сигмоидный выход. Однослойная модель содержит один сигмоидный выходной нейрон. Все веса и смещения инициализированы независимо из $U(-0{,}1;\ 0{,}1)$. Скорость обучения $\eta = 0{,}5$; порог $E_e = 0{,}01$; максимум 20 000 эпох; после каждого обучающего примера обновляются веса.

Формулы: $\sigma(z)=1/(1+\exp(-z))$; $L=\frac{1}{2}(y-t)^2$; $\delta_{выход}=(y-t)\cdot y\cdot(1-y)$; $\delta_{скр}=(W_2\cdot\delta_{выход})\cdot h\cdot(1-h)$. Обновление $W \leftarrow W - \eta\partial L/\partial W$. Суммарная ошибка E_s вычисляется на всех четырёх примерах после завершения каждой эпохи.

Принципиальный методический конфликт: исходные классы −9 и 6 не принадлежат диапазону (0,1) сигмоиды. Поэтому буквальная модель с MSE на исходных целях не может обеспечить $E_s \le 0{,}01$, независимо от наличия скрытого слоя. Для честного сравнения отдельно реализована диагностическая нормализация $t_n=(t+9)/15$ с обратным переводом $y=-9+15y_n$. Эта дополнительная конфигурация явно отделена от буквального условия.

3. Результаты обучения

Конфигурация	Эпохи	E_s	Accuracy	MAE	$E_s \le E_e$
MLP raw targets	20000	117.000036	50%	7.5000	нет
Single raw targets	20000	111.500902	50%	7.2500	нет
MLP scaled diagnostic	2769	0.009998	100%	1.0467	да
Single scaled diagnostic	20000	0.500044	75%	7.5000	нет

Для буквального варианта MLP ошибка осталась приблизительно 117, а у однослойного персептрона — 111,5. Обе модели показали 50% точности при выборе ближайшего числового класса. После нормализации многослойная сеть достигла $E_s \approx 0{,}01$ на

2769-й эпохе и правильно классифицировала все четыре сочетания; однослойная сеть не достигла порога.

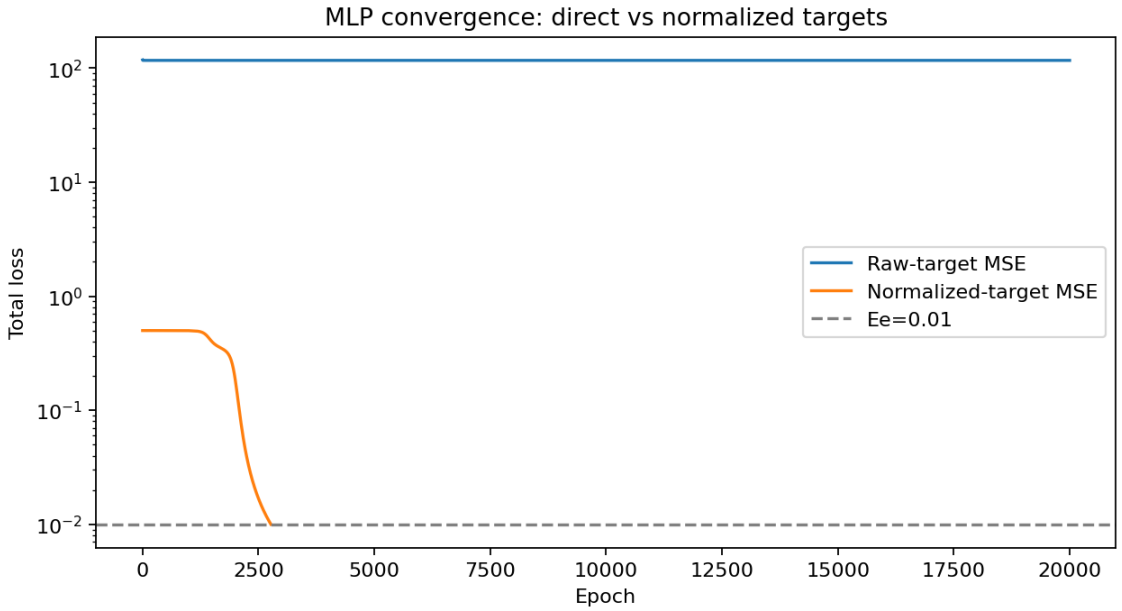


Рисунок 1 — Обучение MLP при разных шкалах целевых значений

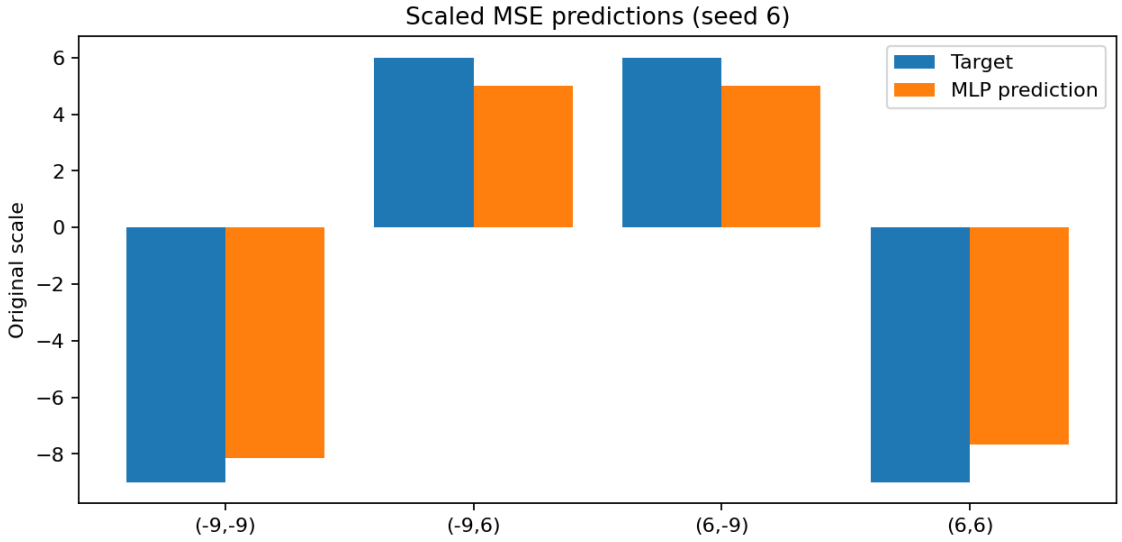


Рисунок 2 — Восстановление исходной шкалы многослойным персептроном

4. Режим функционирования — 8 примеров

A	B	$\hat{y}_n$	ŷ исходное	Ближайший класс
-9	-9	0.0571	-8.1430	-9
-9	6	0.9331	4.9969	6
6	-9	0.9331	4.9967	6
6	6	0.0882	-7.6765	-9
-6	4	0.9200	4.7998	6
0	0	0.8684	4.0254	6
6	0	0.3694	-3.4587	-9
-10	10	0.8684	4.0259	6

Входы для предсказания нормализуются по −9 и 6; выход sigmoid переводится обратно в [−9; 6]. Проверены четыре обучающих сочетания и четыре дополнительных пары из допустимого диапазона [−10; 10]. Для точек вне обучающей таблицы истинности классификацию следует трактовать как обобщение модели, а не как заданную эталонную метку.

5. Выводы по вопросам методички

1) Однослойный персептрон не реализует XOR: его линейная граница не может разнести противоположные вершины таблицы истинности по классам. В нормализованном тесте критерий остановки не достигнут.

2) Многослойная модель способна решить XOR, что видно после математически необходимого приведения целей к допустимой шкале: 2769 эпох, accuracy 100%. При прямых целях −9/6 она не способна достичь порога из-за ограниченности сигмоиды.

3) Произвольные числовые классы вне (0,1) требуют согласования шкалы цели и диапазона выходной активации. Вариант решения — нормализация целей с обратным пересчётом; другой возможный выход — линейная активация для регрессии.

4) Скрытый слой преобразует пространство признаков так, что XOR становится разделимым выходным нейроном; один нейрон с аффинным входом этого не делает.

5) По геометрической сложности кодирование −9/6 не отличается от 0/1: четыре точки остаются XOR. Существенно меняется лишь численная шкала обучения и пригодность выходной сигмоиды.

Листинг программы (полный исходный код)

В архиве код также сохранён отдельными редактируемыми файлами Python; ниже приведён полный листинг без скрытых библиотек машинного обучения.

Файл: xor_engine.py

```
001: """MLP 2-2-1 and single layer perceptron. NumPy is the only dependency.
002: All losses are totals over the four training examples, evaluated after an epoch.
003: """
004: import numpy as np
005:
006: C0, C1 = -9.0, 6.0
007: X_RAW = np.array([[-9.,-9.],[-9.,6.],[6.,-9.],[6.,6.]])
008: Y_RAW = np.array([-9.,6.,6.,-9.])
009: X = (X_RAW - C0) / (C1-C0)
010: Y = (Y_RAW - C0) / (C1-C0)
011:
012: def sigmoid(z):
013:     z = np.clip(z, -60, 60)
014:     return 1 / (1+np.exp(-z))
015:
016: def relu(z):
017:     return np.maximum(z, 0.)
018:
019: class MLP:
020:     def __init__(self, seed=7, activation='sigmoid', loss='bce', lr=1.0):
021:         rng=np.random.RandomState(seed)
022:         self.w1=rng.uniform(-.1,.1,(2,2))
023:         self.b1=rng.uniform(-.1,.1,2)
024:         self.w2=rng.uniform(-.1,.1,2)
025:         self.b2=float(rng.uniform(-.1,.1))
026:         self.activation,self.loss,self.lr=activation,loss,lr
027:
028:     def forward(self, x):
```

```
        z1=x@self.w1+self.b1
        h=sigmoid(z1) if self.activation=='sigmoid' else relu(z1)
        p=sigmoid(h@self.w2+self.b2)
        return z1,h,p

    def predict(self, x):
        return np.asarray(self.forward(np.asarray(x, dtype=float))[2])

    def train_step(self,x,t):
        z,h,p=self.forward(x)
        # BCE + sigmoid: dL/dz = p-t; MSE=1/2*(p-t)^2.
        delta=(p-t) if self.loss=='bce' else (p-t)*p*(1-p)
        dh=(self.w2*delta)*(h*(1-h) if self.activation=='sigmoid' else (z>0))
        old_w2=self.w2.copy()
        self.w2-=self.lr*delta*h
        self.b2-=self.lr*delta
        self.w1-=self.lr*np.outer(x,dh)
        self.b1-=self.lr*dh

    def total_loss(self, xx, yy):
        p=np.clip(self.predict(xx),1e-12,1-1e-12)
        if self.loss=='bce': return float(-np.sum(yy*np.log(p)+(1-yy)*np.log1p(-p)))
        return float(.5*np.sum((p-yy)**2))

    def fit(self,xx,yy,max_epochs=20000,ee=.01,shuffle=True):
        rng=np.random.RandomState(42)
        history=[]
        for epoch in range(1,max_epochs+1):
            order=rng.permutation(len(xx)) if shuffle else range(len(xx))
            for i in order:self.train_step(xx[i],yy[i])
            error=self.total_loss(xx,yy)
            history.append(error)
            if error<=ee:break
        return dict(epochs=epoch,Es=error,converged=error<=ee,history=np.asarray(history))

class SinglePerceptron:
    def __init__(self,seed=7,lr=1.):
        rng=np.random.RandomState(seed)
        self.w=rng.uniform(-.1,.1,2)
        self.b=float(rng.uniform(-.1,.1))
        self.lr=lr
    def predict(self,xx):return sigmoid(np.asarray(xx)@self.w+self.b)
    def fit(self,xx,yy,max_epochs=20000,ee=.01):
        rng=np.random.RandomState(42);h=[]
        for ep in range(1,max_epochs+1):
            for i in rng.permutation(len(xx)):
                x,t=xx[i],yy[i]
                p=float(self.predict(x)); delta=(p-t)*p*(1-p)
                self.w-=self.lr*delta*x;self.b-=self.lr*delta
            p=self.predict(xx);es=float(.5*np.sum((p-yy)**2));h.append(es)
            if es<=ee:break
        return dict(epochs=ep,Es=es,converged=es<=ee,history=np.asarray(h))

def stats(model, raw_output=False):
    out=model.predict(X)
    if raw_output:
        yr=out;cls=np.where(abs(yr-C0)<=abs(yr-C1),C0,C1)
    else:
        yr=C0+(C1-C0)*out;cls=np.where(out<.5,C0,C1)
    return {'accuracy':float(np.mean(cls==Y_RAW)), 'mae':float(np.mean(abs(yr-Y_RAW))),
            'pred_raw':np.asarray(yr).tolist(),'pred_norm':np.asarray(out).tolist(),
            'pred_class':np.asarray(cls).tolist()}

def inference(model,a,b,raw_output=False):
    if not (-10<=a<=10 and -10<=b<=10):raise ValueError('A and B must be in [-10,10]')
    pn=float(model.predict((np.array([a,b])-C0)/(C1-C0)))
    val=pn if raw_output else C0+15*pn
    cls=C0 if abs(val-C0)<=abs(val-C1) else C1
    return {'A':a,'B':b,'output_norm':pn,'output_raw':val,'class':cls}

TEST_PAIRS=[(-9,-9),(-9,6),(6,-9),(6,6),(-6,4),(0,0),(6,0),(-10,10)]
```

Файл: main.py

```
"""Lab 1 (variant 6). Strict specification and a separately labeled scaled diagnostic."""
import json, argparse
from pathlib import Path
from xor_engine import MLP,SinglePerceptron,X,Y,Y_RAW,stats,inference,TEST_PAIRS

```

```
def main(max_epochs=20000, interactive=False):
    models=[('MLP raw targets',MLP(seed=6,loss='mse',lr=.5),Y_RAW,True),
            ('Single raw targets',SinglePerceptron(seed=6,lr=.5),Y_RAW,True),
            ('MLP scaled diagnostic',MLP(seed=6,loss='mse',lr=.5),Y,False),
            ('Single scaled diagnostic',SinglePerceptron(seed=6,lr=.5),Y,False)]
    outputs={}
    for name,model,t,raw in models:
        fit=model.fit(X,t,max_epochs=max_epochs,ee=.01)
        final=stats(model,raw_output=raw)
        outputs[name]={**{k:v for k,v in fit.items() if k!='history'},**final,
            'tests':[inference(model,a,b,raw) for a,b in TEST_PAIRS]}
        print(name,'epochs=',fit['epochs'],'Es=',round(fit['Es'],6),
              'accuracy=',final['accuracy'],'converged=',fit['converged'])
    (Path(__file__).parent/'results').mkdir(exist_ok=True)
    with open(Path(__file__).parent/'results'/'latest_run.json','w',encoding='utf8') as f:
        json.dump(outputs,f,ensure_ascii=False,indent=2)
    if interactive:
        print('Enter A B, or q to quit')
        while True:
            line=input('A B > ').strip()
            if line.lower()=='q':break
            try:
                a,b=map(float,line.split())
                print(inference(models[2][1],a,b))
            except (ValueError,IndexError) as e:print('Input error:',e)

if __name__=='__main__':
    ap=argparse.ArgumentParser();ap.add_argument('--quick',action='store_true')
    ap.add_argument('--interactive',action='store_true');args=ap.parse_args()
    main(1000 if args.quick else 20000,args.interactive)
```